Data Engineering Practicals

Practicals 1 – 10

NAME: Mansi Sharma

STUDENT ID: 5827410

ROLL NO: 68

SUBJECT: Data Engineering

Index

No.	Practical	File name
Practical 1	Noise Elimination, Feature Selection & Exploratory Data Analysis	practical_01_eda.py
Practical 2	Designing a Relational Database and Performing CRUD using SQL	practical_02_sql_crud.py
Practical 3	CRUD Operations in MongoDB (NoSQL)	practical_03_mongodb.js
Practical 4	Extracting Data from APIs and Flat Files	practical_04_data_extraction.py
Practical 5	Setting up Apache Airflow and Creating a DAG	practical_05_airflow_dag.py
Practical 6	ETL Pipeline Design (CSV/JSON, Validation & Incremental Load)	practical_06_etl_pipeline.py
Practical 7	PySpark – Schema, Filtering, Grouping, Deduplication, Joins & Aggregation	practical_07_pyspark.py
Practical 8	End-to-End E-commerce ETL Pipeline (CSV → Pandas → SQL → Report)	practical_08_ecommerce_etl.py
Practical 9	Mini Project – End-to-End Data Engineering Solution (Individual)	practical_09_mini_project_individual.py
Practical 10	Mini Project – End-to-End Data Engineering Solution (Group of 2)	practical_10_mini_project_group.py

Practical 1: Noise Elimination, Feature Selection & Exploratory Data Analysis

Aim

To perform noise (outlier) elimination, feature selection, and exploratory data analysis (EDA) on a dataset using Pandas, NumPy, Matplotlib, and Scikit-learn.

Input (Code)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

data = {
    'Age': [20, 21, 22, 23, 24, 100],       # 100 is an outlier (noise)
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]          # Constant feature
}
df = pd.DataFrame(data)
print("Original Dataset:\n", df)

# Noise elimination using the IQR method
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]
print("\nDataset after Noise Elimination:\n", df_clean)

# Feature selection using variance threshold
selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)
selected_columns = df_clean.columns[selector.get_support()]
print("\nSelected Features:", list(selected_columns))

# Exploratory Data Analysis
print("\nDataset Information:")
print(df_clean.info())
print("\nStatistical Summary:\n", df_clean.describe())
print("\nCorrelation Matrix:\n", df_clean.corr())

df_clean.hist(figsize=(8, 6))
plt.suptitle("Histogram of Features")
plt.savefig("histogram.png")

df_clean.boxplot(figsize=(8, 5))
plt.title("Boxplot of Dataset")
plt.savefig("boxplot.png")
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe"
Original Dataset:
    Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
5  100     20          50         1

Dataset after Noise Elimination:
    Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1

Selected Features: ['Age', 'Marks', 'Attendance']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Age         5 non-null      int64
 1   Marks       5 non-null      int64
 2   Attendance  5 non-null      int64
 3   Constant    5 non-null      int64
dtypes: int64(4)
memory usage: 200.0 bytes
None

Statistical Summary:
             Age      Marks  Attendance  Constant
count   5.000000   5.000000     5.00000       5.0
mean   22.000000  85.000000    89.80000       1.0
std     1.581139   7.905694     3.49285       0.0
min    20.000000  75.000000    85.00000       1.0
25%    21.000000  80.000000    88.00000       1.0
50%    22.000000  85.000000    90.00000       1.0
75%    23.000000  90.000000    92.00000       1.0
max    24.000000  95.000000    94.00000       1.0

Correlation Matrix:
                Age     Marks  Attendance  Constant
Age        1.000000  1.000000    0.995893       NaN
Marks      1.000000  1.000000    0.995893       NaN
Attendance 0.995893  0.995893    1.000000       NaN
Constant        NaN       NaN         NaN       NaN
PS D:\Microsoft VS Code>
```

Practical 2: Designing a Relational Database and Performing CRUD using SQL

Aim

To design a relational database for a small application (a Student–Course system) and perform Create, Read, Update, and Delete (CRUD) operations using SQL.

Input (Code)

```
import sqlite3

conn = sqlite3.connect("college.db")
cur = conn.cursor()

# CREATE: tables
cur.executescript("""
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Enrollments;

CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    age INTEGER
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    credits INTEGER
);
CREATE TABLE Enrollments (
    enroll_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER REFERENCES Students(student_id),
    course_id INTEGER REFERENCES Courses(course_id),
    grade TEXT
);
""")

# CREATE: insert records
cur.executemany("INSERT INTO Students (name, email, age) VALUES (?, ?, ?)", [
    ("Asha Rao", "asha@example.com", 20),
    ("Ravi Kumar", "ravi@example.com", 21),
    ("Neha Shah", "neha@example.com", 22),
])
cur.executemany("INSERT INTO Courses (title, credits) VALUES (?, ?)", [
    ("Data Engineering", 4),
    ("Database Systems", 3),
])
cur.executemany("INSERT INTO Enrollments (student_id, course_id, grade) VALUES (?, ?, ?)", [
    (1, 1, "A"), (1, 2, "B+"), (2, 1, "A-"), (3, 2, "B"),
])
conn.commit()

# READ
print("All Students:")
for row in cur.execute("SELECT * FROM Students"):
    print(row)
```

```
print("\nStudents with their enrolled courses:")
cur.execute("""
    SELECT s.name, c.title, e.grade
    FROM Enrollments e
    JOIN Students s ON s.student_id = e.student_id
    JOIN Courses c ON c.course_id = e.course_id
""")
for row in cur.fetchall():
    print(row)

# UPDATE
cur.execute("UPDATE Students SET age = 23 WHERE name = 'Ravi Kumar'")
conn.commit()
print("\nAfter UPDATE:")
for row in cur.execute("SELECT * FROM Students WHERE name = 'Ravi Kumar'"):
    print(row)

# DELETE
cur.execute("DELETE FROM Enrollments WHERE grade = 'B'")
conn.commit()
print("\nAfter DELETE (grade='B' rows removed):")
for row in cur.execute("SELECT * FROM Enrollments"):
    print(row)

conn.close()
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe"
All Students:
(1, 'Asha Rao', 'asha@example.com', 20)
(2, 'Ravi Kumar', 'ravi@example.com', 21)
(3, 'Neha Shah', 'neha@example.com', 22)

Students with their enrolled courses:
('Asha Rao', 'Data Engineering', 'A')
('Asha Rao', 'Database Systems', 'B+')
('Ravi Kumar', 'Data Engineering', 'A-')
('Neha Shah', 'Database Systems', 'B')

After UPDATE:
(2, 'Ravi Kumar', 'ravi@example.com', 23)

After DELETE (grade='B' rows removed):
(1, 1, 1, 'A')
(2, 1, 2, 'B+')
(3, 2, 1, 'A-')
PS D:\Microsoft VS Code>
```

Practical 3: CRUD Operations in MongoDB (NoSQL)

Aim

To perform single and multiple document insertion and retrieval using the MongoDB shell (mongosh), as a NoSQL counterpart to the relational CRUD in Practical 2.

Input (Query (mongosh))

```
// Step 1: Insert a single document
db.items.insertOne({ name: "laptop", price: 999 });

// Step 2: View the inserted document
db.items.find();

// Step 3: Insert multiple documents
db.products.insertMany([
    { name: "phone", price: 500, stock: 10 },
    { name: "tablet", price: 300, stock: 5 },
    { name: "watch", price: 150, stock: 0 }
]);

// Step 4: View products in formatted JSON
db.products.find().pretty();

// Step 5: Update a document
db.products.updateOne({ name: "watch" }, { $set: { stock: 25 } });

// Step 6: Delete a document
db.products.deleteOne({ name: "table// Step 1: Insert a single document
db.items.insertOne({ name: "laptop", price: 999 });

// Step 2: View the inserted document
db.items.find();

// Step 3: Insert multiple documents
db.products.insertMany([
    { name: "phone", price: 500, stock: 10 },
    { name: "tablet", price: 300, stock: 5 },
    { name: "watch", price: 150, stock: 0 }
]);

// Step 4: View products in formatted JSON
db.products.find().pretty();

// Step 5: Update a document
db.products.updateOne({ name: "watch" }, { $set: { stock: 25 } });

// Step 6: Delete a document
db.products.deleteOne({ name: "tablet" });
t" });
// Step 1: Insert a single document
db.items.insertOne({ name: "laptop", price: 999 });

// Step 2: View the inserted document
db.items.find();

// Step 3: Insert multiple documents
db.products.insertMany([
```

```
    { name: "phone", price: 500, stock: 10 },
    { name: "tablet", price: 300, stock: 5 },
    { name: "watch", price: 150, stock: 0 }
]);

// Step 4: View products in formatted JSON
db.products.find().pretty();

// Step 5: Update a document
db.products.updateOne({ name: "watch" }, { $set: { stock: 25 } });

// Step 6: Delete a document
db.products.deleteOne({ name: "tablet" });
```

mongosh - MongoDB Shell

```
test> // Step 1: Insert a single document
test> db.items.insertOne({ name: "laptop", price: 999 });
{
  acknowledged: true,
  insertedId: ObjectId('6a4f16899c942391a0098a7b')
}

test> // Step 2: View the inserted document
test> db.items.find();
[
  {
    _id: ObjectId('6a4f16899c942391a0098a7b'),
    name: 'laptop',
    price: 999
  }
]

test> // Step 3: Insert multiple documents
test> db.products.insertMany([
...     { name: "phone", price: 500, stock: 10 },
...     { name: "tablet", price: 300, stock: 5 },
...     { name: "watch", price: 150, stock: 0 }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a4f16d09c942391a0098a7c'),
    '1': ObjectId('6a4f16d09c942391a0098a7d'),
    '2': ObjectId('6a4f16d09c942391a0098a7e')
  }
}

test> // Step 4: View products in formatted JSON
test> db.products.find().pretty();
[
  {
    _id: ObjectId('6a4f16d09c942391a0098a7c'),
    name: 'phone',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('6a4f16d09c942391a0098a7d'),
    name: 'tablet',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6a4f16d09c942391a0098a7e'),
    name: 'watch',
    price: 150,
    stock: 0
  }
]

test> // Step 5: Update a document
test> db.products.updateOne({ name: "watch" }, { $set: { stock: 25 } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

test> // Step 6: Delete a document
test> db.products.deleteOne({ name: "tablet" });
{ acknowledged: true, deletedCount: 1 }
test>
```

Practical 4: Extracting Data from APIs and Flat Files

Aim

To fetch user data from a public REST API, clean it, and merge it with a flat CSV file containing location details.

Input (Code (data_extraction.py))

```
import pandas as pd
import requests



def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        return pd.json_normalize(data)   # Normalize JSON into a flat table
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()



def extract_csv_data(file_path):
    """Reads a flat CSV file."""
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()



if __name__ == "__main__":
    api_url = "https://jsonplaceholder.typicode.com/users"
    api_df = extract_api_data(api_url)
    api_df = api_df[["id", "name", "email", "company.name"]]
    api_df.rename(columns={"company.name": "company"}, inplace=True)

    csv_file = "locations.csv"
    pd.DataFrame({
        "id": list(range(1, 11)),
        "city": ["New York", "London", "Paris", "Tokyo", "Berlin",
                 "Delhi", "Sydney", "Moscow", "Cairo", "Beijing"],
    }).to_csv(csv_file, index=False)
    csv_df = extract_csv_data(csv_file)

    if not api_df.empty and not csv_df.empty:
        merged_df = pd.merge(api_df, csv_df, on="id", how="inner")
        print("\n--- Merged ETL Pipeline Data View ---")
        print(merged_df.head())
        merged_df.to_csv("cleaned_warehouse_profiles.csv", index=False)
        print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")

        print("\nMissing values per column:\n", merged_df.isnull().sum())
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe" "c:/Users/Ayaan Kh

--- Merged ETL Pipeline Data View ---
   id             name                       email            company      city
0   1     Leanne Graham          Sincere@april.biz     Romaguera-Crona  New York
1   2      Ervin Howell          Shanna@melissa.tv       Deckow-Crist    London
2   3  Clementine Bauch         Nathan@yesenia.net  Romaguera-Jacobson     Paris
3   4  Patricia Lebsack  Julianne.OConner@kory.org       Robel-Corkery     Tokyo
4   5  Chelsey Dietrich   Lucio_Hettinger@annie.ca         Keebler LLC    Berlin

Data successfully saved to 'cleaned_warehouse_profiles.csv'

Missing values per column:
 id         0
name        0
email       0
company     0
city        0
dtype: int64
PS D:\Microsoft VS Code>
```

Practical 5: Setting up Apache Airflow and Creating a DAG

Aim

To configure a Directed Acyclic Graph (DAG) in Apache Airflow that runs daily, executing a start notifier, an extraction script, and a pipeline-success logger in sequence.

Input (Code (~/airflow/dags/data_pipeline_dag.py))

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="Lab assignment for API and Flat File ETL orchestration",
    schedule_interval=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(task_id="start_pipeline")

    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /absolute/path/to/data_extraction.py",
    )

    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)"',
    )

    start_pipeline >> execute_extraction >> pipeline_complete
```

Commands to Run (Terminal)

```
airflow dags list
airflow tasks test university_etl_orchestration start_pipeline 2026-01-01
airflow tasks test university_etl_orchestration run_extraction_script 2026-01-01
airflow tasks test university_etl_orchestration log_pipeline_success 2026-01-01
```

```
student@ubuntu: ~/airflow

$ airflow dags list
dag_id                    | filepath                | owner               | paused
==========================+=========================+=====================+=======
university_etl_orchestration | university_etl_dag.py  | data_engineering_lab | False

$ airflow tasks test university_etl_orchestration start_pipeline 2026-01-01
INFO - Filling up the DagBag from /home/student/airflow/dags
INFO - Dependencies all met for dep_context=non-requeueable deps
INFO - Starting attempt 1 of 2
INFO - Executing <Task(EmptyOperator): start_pipeline> on 2026-01-01T00:00:00+00:00
INFO - Marking task as SUCCESS. dag_id=university_etl_orchestration, task_id=start_pipeline

$ airflow tasks test university_etl_orchestration run_extraction_script 2026-01-01
INFO - Executing <Task(BashOperator): run_extraction_script> on 2026-01-01T00:00:00+00:00
INFO - Running command: ['/usr/bin/bash', '-c', 'python3 /absolute/path/to/data_extraction.py']
INFO - Output:
  ... your data_extraction.py print statements appear here ...
INFO - Command exited with return code 0
INFO - Marking task as SUCCESS. dag_id=university_etl_orchestration, task_id=run_extraction_script

$ airflow tasks test university_etl_orchestration log_pipeline_success 2026-01-01
INFO - Executing <Task(BashOperator): log_pipeline_success> on 2026-01-01T00:00:00+00:00
INFO - Running command: ['/usr/bin/bash', '-c', 'echo "ETL Execution completed successfully at $(date)"']
INFO - Output:
INFO - ETL Execution completed successfully at Mon Jan  1 00:00:00 UTC 2026
INFO - Command exited with return code 0
INFO - Marking task as SUCCESS. dag_id=university_etl_orchestration, task_id=log_pipeline_success
```

Example of expected output only, not a real run.

Practical 6: ETL Pipeline Design (CSV/JSON, Validation & Incremental Load)

Aim

To design an ETL pipeline that (a) extracts data from multiple CSV files and combines them, (b) extracts JSON data and loads selected fields into a relational database, (c) removes invalid records, (d) validates data before loading, and (e) supports incremental loading.

Input (Code)

```
import pandas as pd
import sqlite3
import json
import os

DB_FILE = "warehouse.db"



def extract_multiple_csv(file_list):
    frames = [pd.read_csv(f) for f in file_list if os.path.exists(f)]
    return pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()



def extract_json(file_path):
    with open(file_path) as f:
        raw = json.load(f)
    df = pd.json_normalize(raw)
    return df[["order_id", "customer_name", "amount"]]



def clean_invalid(df):
    df = df.dropna(subset=["order_id", "amount"])
    df = df[df["amount"] > 0]
    df = df.drop_duplicates(subset="order_id")
    return df



def validate(df, required_cols):
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        raise ValueError(f"Validation failed - missing columns: {missing}")
    if df.empty:
        raise ValueError("Validation failed - no records to load")
    print(f"Validation passed: {len(df)} records ready to load.")
    return True



def incremental_load(df, table_name, key_col):
    conn = sqlite3.connect(DB_FILE)
    try:
        existing_ids = pd.read_sql(
            f"SELECT {key_col} FROM {table_name}", conn)[key_col].tolist()
    except Exception:
        existing_ids = []
    new_rows = df[~df[key_col].isin(existing_ids)]
    if not new_rows.empty:
        new_rows.to_sql(table_name, conn, if_exists="append", index=False)
```

```
    print(f"Incremental load: {len(new_rows)} new record(s) inserted "
          f"(skipped {len(df) - len(new_rows)} already-loaded record(s)).")
    conn.close()


if __name__ == "__main__":
    # Sample source files
    jan = pd.DataFrame({"order_id": [1, 2], "customer_name": ["A", "B"],
                        "amount": [100, 200]})
    feb = pd.DataFrame({"order_id": [3, 4], "customer_name": ["C", "D"],
                        "amount": [-50, 300]})
    jan.to_csv("orders_jan.csv", index=False)
    feb.to_csv("orders_feb.csv", index=False)

    combined = extract_multiple_csv(["orders_jan.csv", "orders_feb.csv"])
    print("Combined CSV data:\n", combined)

    cleaned = clean_invalid(combined)
    print("\nAfter removing invalid records:\n", cleaned)

    validate(cleaned, ["order_id", "customer_name", "amount"])

    incremental_load(cleaned, "orders", "order_id")   # 1st run: inserts
    incremental_load(cleaned, "orders", "   order_id")   # 2nd run: skips
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe" "c:/Users/Ayaan
Combined CSV data:
    order_id customer_name  amount
0         1             A     100
1         2             B     200
2         3             C     -50
3         4             D     300

After removing invalid records:
    order_id customer_name  amount
0         1             A     100
1         2             B     200
3         4             D     300
Validation passed: 3 records ready to load.
Incremental load: 3 new record(s) inserted (skipped 0 already-loaded record(s)).
Incremental load: 0 new record(s) inserted (skipped 3 already-loaded record(s)).
PS D:\Microsoft VS Code>
```

Practical 7: PySpark – Schema, Filtering, Grouping, Deduplication, Joins & Aggregation

Aim

To read a CSV dataset in PySpark, display its schema, perform filtering/grouping/aggregation, remove duplicate records, join two datasets, and calculate average sales by product category.

Input (Code)

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("DataEngineeringLab").getOrCreate()

sales_data = [
    (1, "Laptop", "Electronics", 55000, 2),
    (2, "Phone", "Electronics", 20000, 5),
    (3, "Phone", "Electronics", 20000, 5),    # duplicate row
    (4, "Shirt", "Clothing", 800, 10),
    (5, "Jeans", "Clothing", 1500, 4),
    (6, "Sofa", "Furniture", 25000, 1),
]
columns = ["order_id", "product", "category", "price", "quantity"]
sales_df = spark.createDataFrame(sales_data, columns)

# Write to CSV and read it back
sales_df.write.mode("overwrite").option("header", True).csv("sales_csv")
csv_df = spark.read.option("header", True).option("inferSchema", True).csv("sales_csv")

# Display schema
csv_df.printSchema()

# Filtering
high_value = csv_df.filter(csv_df.price > 1000)
high_value.show()

# Grouping and aggregation
category_totals = csv_df.groupBy("category").agg(
    F.sum(csv_df.price * csv_df.quantity).alias("total_sales")
)
category_totals.show()

# Remove duplicates
deduped = csv_df.dropDuplicates(["product", "category", "price", "quantity"])
deduped.show()

# Join with a second dataset
discounts = spark.createDataFrame(
    [("Electronics", 10), ("Clothing", 20), ("Furniture", 5)],
    ["category", "discount_pct"]
)
joined = deduped.join(discounts, on="category", how="left")
joined.show()

# Average sales by category
avg_sales = deduped.groupBy("category").agg(
    F.avg(deduped.price * deduped.quantity).alias("avg_sales")
```

```
)
avg_sales.show()

spark.stop()
```

```
root
 |-- order_id: integer (nullable = true)
 |-- product: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: integer (nullable = true)
 |-- quantity: integer (nullable = true)

+--------+-------+-----------+-----+--------+
|order_id|product|   category|price|quantity|
+--------+-------+-----------+-----+--------+
|       1| Laptop|Electronics|55000|       2|
|       2|  Phone|Electronics|20000|       5|
|       3|  Phone|Electronics|20000|       5|
|       5|  Jeans|   Clothing| 1500|       4|
|       6|   Sofa|  Furniture|25000|       1|
+--------+-------+-----------+-----+--------+

+-----------+-----------+
|   category|total_sales|
+-----------+-----------+
|Electronics|     310000|
|   Clothing|      14000|
|  Furniture|      25000|
+-----------+-----------+
```

```
+--------+-------+-----------+-----+--------+
|order_id|product|   category|price|quantity|
+--------+-------+-----------+-----+--------+
|       1| Laptop|Electronics|55000|       2|
|       2|  Phone|Electronics|20000|       5|
|       6|   Sofa|  Furniture|25000|       1|
|       4|  Shirt|   Clothing|  800|      10|
|       5|  Jeans|   Clothing| 1500|       4|
+--------+-------+-----------+-----+--------+

+-----------+--------+-------+-----+--------+------------+
|   category|order_id|product|price|quantity|discount_pct|
+-----------+--------+-------+-----+--------+------------+
|Electronics|       1| Laptop|55000|       2|          10|
|Electronics|       2|  Phone|20000|       5|          10|
|   Clothing|       4|  Shirt|  800|      10|          20|
|   Clothing|       5|  Jeans| 1500|       4|          20|
|  Furniture|       6|   Sofa|25000|       1|           5|
+-----------+--------+-------+-----+--------+------------+

+-----------+---------+
|   category|avg_sales|
+-----------+---------+
|Electronics| 105000.0|
|   Clothing|   7000.0|
|  Furniture|  25000.0|
+-----------+---------+
```

Practical 8: End-to-End E-commerce ETL Pipeline (CSV → Pandas → SQL → Report)

Aim

To build an end-to-end ETL pipeline for an e-commerce dataset (customers, orders, products, payments), producing a cleaned SQL database and a summary report, while handling both historical and newly-arriving data.

Input (Code)

```
import pandas as pd
import sqlite3
from datetime import datetime

DB_FILE = "ecommerce.db"

# ---------- EXTRACT ----------
customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "name": ["Asha", "Ravi", "Neha"],
    "city": ["Mumbai", "Delhi", "Pune"],
})
products = pd.DataFrame({
    "product_id": [101, 102, 103],
    "product_name": ["Laptop", "Phone", "Headphones"],
    "category": ["Electronics", "Electronics", "Accessories"],
})
orders_historical = pd.DataFrame({
    "order_id": [1, 2],
    "customer_id": [1, 2],
    "product_id": [101, 102],
    "order_date": ["2026-01-05", "2026-01-06"],
    "quantity": [1, 2],
})
orders_new = pd.DataFrame({
    "order_id": [3, 4],
    "customer_id": [3, 1],
    "product_id": [103, 102],
    "order_date": ["2026-01-20", "2026-01-21"],
    "quantity": [3, 1],
})
payments = pd.DataFrame({
    "order_id": [1, 2, 3, 4],
    "amount": [55000, 40000, 4500, 20000],
    "status": ["Paid", "Paid", "Pending", "Paid"],
})



# ---------- TRANSFORM ----------
def clean(df):
    return df.drop_duplicates().dropna()



orders_all = pd.concat([orders_historical, orders_new], ignore_index=True)
orders_all["order_date"] = pd.to_datetime(orders_all["order_date"])

merged = (clean(orders_all)
          .merge(clean(customers), on="customer_id", how="left")
```

```
        .merge(clean(products), on="product_id", how="left")
        .merge(clean(payments), on="order_id", how="left"))

# ---------- LOAD ----------
conn = sqlite3.connect(DB_FILE)
merged.to_sql("fact_orders", conn, if_exists="replace", index=False)

# ---------- REPORT ----------
report = pd.read_sql("""
    SELECT city, category, SUM(amount) AS revenue, COUNT(*) AS orders
    FROM fact_orders
    WHERE status = 'Paid'
    GROUP BY city, category
    ORDER BY revenue DESC
""", conn)
print("Revenue Report (Paid Orders):\n", report)

conn.close()
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe" "
Revenue Report (Paid Orders):
      city     category  revenue  orders
0  Mumbai  Electronics    75000       2
1   Delhi  Electronics    40000       1
PS D:\Microsoft VS Code>
```

Practical 9: Mini Project – End-to-End Data Engineering Solution (Individual)

Aim

To design and implement a complete data engineering solution covering ingestion, cleaning, transformation, storage, data-warehouse design, and reporting for a retail sales dataset.

Architecture: Raw CSV/API sources → Staging (Pandas) → Cleaning & Transformation → Star-schema Data Warehouse (SQLite) → Reporting layer (SQL views / charts)

Input (Code)

```
import pandas as pd
import sqlite3

DB_FILE = "retail_dw.db"

# ---------- INGEST ----------
sales_raw = pd.DataFrame({
    "txn_id": [1, 2, 3, 4, 5],
    "date": ["2026-02-01", "2026-02-01", "2026-02-02", "2026-02-02", None],
    "store": ["Mumbai", "Pune", "Mumbai", "Delhi", "Pune"],
    "product": ["Laptop", "Phone", "Laptop", "Tablet", "Phone"],
    "amount": [55000, 20000, 55000, None, 21000],
})

# ---------- CLEAN ----------
sales = sales_raw.dropna(subset=["date", "amount"]).drop_duplicates(subset="txn_id").copy()
sales["date"] = pd.to_datetime(sales["date"])

# ---------- TRANSFORM (Star Schema) ----------
dim_store = sales[["store"]].drop_duplicates().reset_index(drop=True)
dim_store["store_id"] = dim_store.index + 1

dim_product = sales[["product"]].drop_duplicates().reset_index(drop=True)
dim_product["product_id"] = dim_product.index + 1

fact_sales = (sales
              .merge(dim_store, on="store")
              .merge(dim_product, on="product"))[
    ["txn_id", "date", "store_id", "product_id", "amount"]]

# ---------- LOAD ----------
conn = sqlite3.connect(DB_FILE)
dim_store.to_sql("dim_store", conn, if_exists="replace", index=False)
dim_product.to_sql("dim_product", conn, if_exists="replace", index=False)
fact_sales.to_sql("fact_sales", conn, if_exists="replace", index=False)

# ---------- REPORT ----------
report = pd.read_sql("""
    SELECT s.store, SUM(f.amount) AS total_revenue, COUNT(*) AS transactions
    FROM fact_sales f
    JOIN dim_store s ON s.store_id = f.store_id
    GROUP BY s.store
    ORDER BY total_revenue DESC
""", conn)
print("Store-wise Revenue Report:\n", report)
```

```
conn.close()
```

```
PS D:\Microsoft VS Code> & "C:\Program Files\Python312\python.exe"
Store-wise Revenue Report:
     store  total_revenue  transactions
0  Mumbai       110000.0             2
1    Pune        20000.0             1
PS D:\Microsoft VS Code>
```

Practical 10: Mini Project – End-to-End Data Engineering Solution (Group of 2)

Aim

As a group of two, design and implement a complete data-engineering solution — data ingestion, cleaning, transformation, storage, data-warehouse design, and reporting — for a larger, jointly-chosen dataset, dividing responsibilities between both members.

Project Plan & Role Split

Team Member A – Ingestion & Cleaning

- Source the dataset (CSV / public API / Kaggle dataset)
- Write ingestion scripts (extract_api.py / extract_csv.py)
- Handle missing values, duplicates, and data-type corrections
- Output: cleaned_data.csv

Team Member B – Transformation, Warehouse & Reporting

- Design a star schema (fact + dimension tables) from the cleaned data
- Load into a database (SQLite / PostgreSQL) using the incremental-load pattern from Practical 6
- Build the reporting layer: SQL views / a small Pandas + Matplotlib or Power BI dashboard summarizing key metrics

Joint Work

- Orchestrate the full pipeline with an Airflow DAG (as in Practical 5) chaining: ingest → clean → transform → load → report
- Document the architecture diagram and prepare the final presentation

Input (Orchestration Code (Joint))

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator



def ingest():
    print("Member A: extracting and cleaning raw data...")



def transform_load():
    print("Member B: transforming into star schema and loading warehouse...")



def report():
    print("Joint: generating final summary report...")



default_args = {
    "owner": "team",
    "start_date": datetime(2026, 1, 1),
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}
```

```
with DAG("group_mini_project_pipeline", default_args=default_args,
         schedule_interval="@daily", catchup=False) as dag:

    t1 = PythonOperator(task_id="ingest_and_clean", python_callable=ingest)
    t2 = PythonOperator(task_id="transform_and_load", python_callable=transform_load)
    t3 = PythonOperator(task_id="generate_report", python_callable=report)

    t1 >> t2 >> t3
```

Deliverables for submission: cleaned dataset, warehouse schema diagram (fact/dimension tables), the Airflow DAG script, the reporting output (table/chart/dashboard screenshot), and a short write-up describing each member's contribution.

